Algoritmos II, Trabalho Prático 1 - Indexação Geoespacial e Geometria Computacional Híbrida

Samuel H. R. de Almeida - 2024066083; Laércio S. R. B. Neto - 2024066105

Departamento de Ciência da Computação – Universidade Federal de Minas Gerais (UFMG) – Belo Horizonte - MG - Brasil

{samuelhigino, laercioborba}@dcc.ufmg.br

1. Introdução

O objetivo do trabalho é desenvolver um mapa interativo de bares e restaurantes de Belo Horizonte, permitindo ao usuário criar entregadores fictícios de um aplicativo de delivery ao redor dos restaurantes e pesquisar por entregadores próximos a algum restaurante. A interface do programa deve permitir ao usuário inserir uma quantidade arbitrária de entregadores para serem distribuídos uniformemente para cada restaurante, e ao redor de cada um seguindo uma distribuição gaussiana (normal), além de ter um campo de pesquisa para o usuário encontrar um restaurante e filtrar os entregadores a um determinado raio da região.

Este relatório está organizado em 4 seções:

1. Introdução;
2. Implementação: apresentamos as decisões a respeito da implementação do projeto, bem como suas limitações e dificuldades encontradas ao longo do desenvolvimento;
3. Testes: detalhamento sobre testes feitos ao longo do desenvolvimento que consideramos relevantes;
4. Conclusão: resumo geral do processo de desenvolvimento e destaque para os tópicos de Algoritmos II que foram explorados.

2. Decisões de Implementação

2.1. Processamento de Dados

Como dados preliminares, nos foi fornecida uma tabela, em formato csv, de pouco menos de 21.000 linhas contendo endereços e nomes de bares e restaurantes. Para operar com arquivos do tipo csv, utilizamos a biblioteca Pandas (1).

Nos dados brutos do trabalho, as coordenadas dos bares e restaurantes não são fornecidas, então é necessário extrai-las de algum lugar externo. Para isso, optamos por utilizar a API da Geocoding (2), por ser uma das poucas opções de API que permitem fazer mais de uma consulta por segundo com uma conta gratuita. Mais especificamente, a Geocoding permite fazer 5 consultas por segundo por um máximo de 25 mil consultas, e, como para o trabalho precisamos de pouco menos de 21 mil coordenadas, ela se encaixa dentro deste limite.

Mesmo assim, por ser uma API e permitir 5 consultas por segundo, a extração dos dados é demorada – leva entre 4 a 6 horas – e isso é uma limitação. Ao menos, a extração teve de ser feita apenas uma vez e salva em um outro arquivo, então o passo de processamento de dados pode ser pulado pelo usuário e pode-se apenas usar a base nova. Além disso, algumas consultas podem falhar por diversos motivos, e isso resultou com que tivéssemos a perda de alguns endereços que não puderam ser encontrados pela API, apesar de esta perda representar menos de 1% dos dados originais.

2.2. Geohash

A parte relativa ao *geohash* foi implementada em um módulo contendo duas funções: uma que recebe um par de coordenadas e traduz para um *geohash* de nível arbitrário (por padrão 8), e outra que recebe um *geohash* de qualquer tamanho e o traduz para coordenadas. Obviamente, pelo funcionamento do *geohash*, sempre há algum nível de perda de informação quando codificamos uma coordenada para geohash e quando a decodificamos, mas esta limitação é inevitável.

2.3. Trie

Antes da implementação da *Trie* em si, no mesmo módulo, implementamos uma classe *Node* para os nós da *Trie*, que armazena o prefixo da subárvore da qual o nó é a raíz, um dicionário de chaves que mapeiam para outros nós filhos do nó, uma variável booleana que representa se o nó guarda a informação de uma palavra que existe na árvore e o índice da palavra que ele armazena no vetor da Trie (-1, caso ele não armazene palavra). A classe *Node* só possui métodos que foram necessários para algum processo da *Trie*.

A *Trie* em si também é implementada em uma classe, com um nó raíz de referência que contém uma palavra vazia, uma lista de palavras que estão contidas nela e um contador de quantas palavras ela armazena. Além das funções padrão de inserir uma palavra e pesquisar por uma palavra na árvore, também implementamos uma função de pesquisar por um prefixo, que retorna todas as palavras da árvore que contém aquele prefixo e uma função para esvaziar a árvore. Decidimos não implementar uma função de remover apenas uma palavra da árvore pois não seria necessário em nenhum momento para o projeto.

2.4. Geração de Entregadores

A geração dos entregadores foi implementada com a biblioteca NumPy (4), utilizando uma distribuição normal para gerar posições próximas aos restaurantes selecionados aleatoriamente. Como essas posições podem não corresponder a ruas, foi utilizado o OSMnx (3) para projetá-las sobre as arestas mais próximas da rede viária de Belo Horizonte.

2.5. Interface e Mapa Interativo

A interface foi desenvolvida com Dash, utilizando Plotly para exibir os restaurantes e entregadores em um mapa interativo. O usuário pode selecionar um restaurante, gerar entregadores e definir um raio de proximidade. Por padrão, limitamos o número de

entregadores gerados a 10.000, devido ao alto tempo de latência para um número muito grande de entregadores.

Para otimizar as buscas, indexamos os entregadores por células Geohash. A partir das células próximas ao restaurante selecionado, obtemos candidatos e utilizamos a fórmula de Haversine para verificar quais estão efetivamente dentro do raio definido.

3. Testes

Testamos a codificação e decodificação de Geohash, a geração e projeção dos entregadores e a filtragem por diferentes restaurantes e raios. Também verificamos a atualização do mapa sem gerar novamente os entregadores a cada alteração dos filtros. Além, de casos de borda nos inputs dos elementos presentes no HTML. Esses testes permitiram avaliar a integração e o funcionamento dos principais componentes do sistema. Abaixo, podemos ver uma imagem do programa em funcionamento.

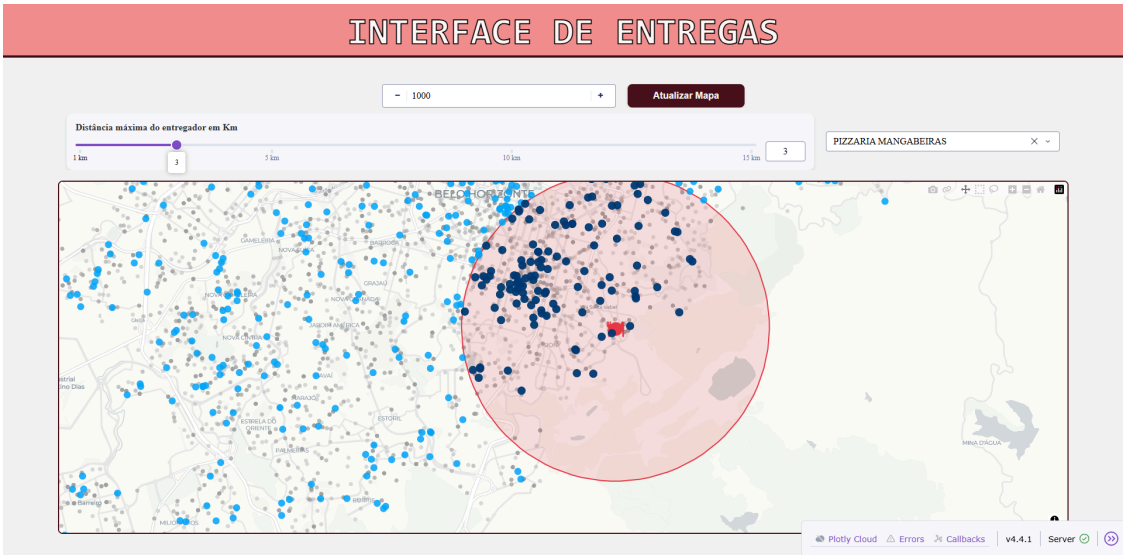


Figura 1. Exemplo de execução do programa, fazendo uma pesquisa em um raio de 3km ao redor da Pizzaria Mangabeiras, tendo gerado 1000 entregadores.

O tempo de resposta para a plotagem dos entregadores foi a maior limitação em tempo do sistema, uma vez que o processo de projetar os entregadores nas vias próximas é muito custoso. No geral, para números pequenos de entregadores (menos de 1.000), o sistema responde em até 5 segundos. Para números maiores de entregadores, o tempo aumenta progressivamente. Já o tempo de cálculo do raio de entregadores próximos é feito muito rapidamente.

4. Conclusão

Este trabalho foi dado com o objetivo de praticar o conteúdo de manipulação de sequências e geometria computacional, dado em sala de aula. No entanto, além destes dois conteúdos, o projeto acabou também envolvendo conhecimentos de operações com dados e interface gráfica.

No fim, foi um trabalho enriquecedor e útil para aplicar o conteúdo visto em sala de aula e ter a experiência de aplicá-lo em um contexto prático, mesmo que fictício. As dificuldades encontradas durante o desenvolvimento nos permitiram ter um momento de parar e pensar sobre a execução de cada algoritmo sendo implementado, o que pode se mostrar bastante útil quando eventualmente aplicarmos estes conhecimentos em outras situações.

Referência Bibliográfica

1. Pandas - Documentação. http://pandas.pydata.org/docs/
2. Geocoding API. https://geocode.maps.co/docs/
3. Boeing, G. (2025). Modeling and Analyzing Urban Networks and Amenities with OSMnx. Geographical Analysis 57 (4), 567-577. doi:10.1111/gean.70009
4. Python 3.15.0 - Documentação. https://docs.python.org/pt-br/3/
5. Wikipedia - Geohash. https://en.wikipedia.org/wiki/Geohash
6. Wikipedia - Radix Tree. https://en.wikipedia.org/wiki/Radix_tree